



**Standardized Python Pipeline for Evaluating Perfusion
Imaging Challenge Submissions**

Google Summer of Code 2026 Proposal

Ranya Al-khleef

Table of Contents

Overview	2
Goals	2
Functional Requirements	2
Non Functional Requirements	3
Tools and Technologies	3
Technical Approach	3
User Flow	4
Backend Flow	4
Workflow Summary	5
Project Structure	5
Deliverables	6
Timeline	6
Risks and Challenges	7
Mitigation Strategies	7

Overview

The OSIPi perfusion imaging challenges evaluate computational methods for generating perfusion parameter maps from MRI data. In current workflows, submissions are often validated, executed, and scored manually using separate scripts. This process can be time consuming, error prone, and inconsistent across teams.

This project proposes a standardized Python pipeline that automates the evaluation process. The system will ingest challenge submissions, validate dataset structure and file formats, execute submitted methods in a controlled environment, compute evaluation metrics against reference data, and generate structured reports for comparison. The goal is to improve reproducibility, consistency, and efficiency in benchmarking perfusion imaging methods.

Goals

- Build a modular Python pipeline for handling challenge submissions
- Automate validation of dataset formats such as NIfTI, BIDS, and required naming conventions
- Execute submitted code in an isolated environment using Docker
- Compute evaluation metrics such as RMSE, accuracy, repeatability, and reproducibility
- Generate structured outputs including tables, rankings, and visualizations

Functional Requirements

- The system shall accept submission packages containing parameter maps, metadata, and associated code.
- The system shall ingest submissions from a structured input directory or uploaded archive.
- The system shall validate submitted datasets to ensure correct file formats and required files are present.
- The system shall verify that dataset organization follows expected standards such as BIDS or another defined structure.
- The system shall enforce naming conventions for submitted files.
- The system shall reject invalid submissions and provide clear error messages.
- The system shall execute submitted code in an isolated environment using Docker.
- The system shall capture execution logs and errors during runtime.
- The system shall compare generated outputs against reference datasets.
- The system shall compute evaluation metrics including RMSE, accuracy, repeatability, and reproducibility measures.
- The system shall aggregate results across submissions for comparison.
- The system shall generate structured output files such as CSV tables containing evaluation results.
- The system shall produce rankings of submitted methods based on evaluation metrics.
- The system shall generate visualizations to support comparison of results.
- The system shall generate a summary report in HTML or PDF format containing evaluation outcomes.

Non Functional Requirements

- The system shall ensure reproducibility of results through controlled execution environments.
- The system shall be modular, with separate components for submission handling, validation, execution, evaluation, and reporting.
- The system shall be scalable to handle multiple submissions and large imaging datasets.
- The system shall provide reliable execution with proper error handling and logging.
- The system shall provide clear and interpretable outputs for users.
- The system shall maintain reasonable performance when processing large datasets.
- The system shall be maintainable and extensible to support additional datasets, metrics, and workflows.
- The system shall ensure security by isolating execution of submitted code.

Tools and Technologies

- Python as the primary programming language
- Docker for isolated and reproducible execution environments
- NIfTI as the primary imaging file format
- BIDS or another defined dataset structure for organizing submissions
- NumPy and SciPy for scientific computing and evaluation
- nibabel for reading and validating NIfTI imaging files
- pandas for structured results and tabular outputs
- Matplotlib for visualizations
- GitHub for version control and collaboration
- GitHub Actions for testing and validation support

Technical Approach

The pipeline will be designed as a modular system with five main components:

Input and Submission Handling

Participants will submit structured folders containing parameter maps, metadata, and the code used to generate results. The pipeline will ingest these submissions from a defined input directory or uploaded archive and prepare them for validation.

Validation Module

The validation module will verify file formats, dataset organization, required files, and naming conventions. Invalid submissions will be flagged with clear error messages before execution begins.

Execution Module

Submitted workflows will be executed in an isolated Docker environment to ensure reproducibility and prevent system conflicts. Execution logs, runtime messages, and errors will be recorded for transparency and debugging.

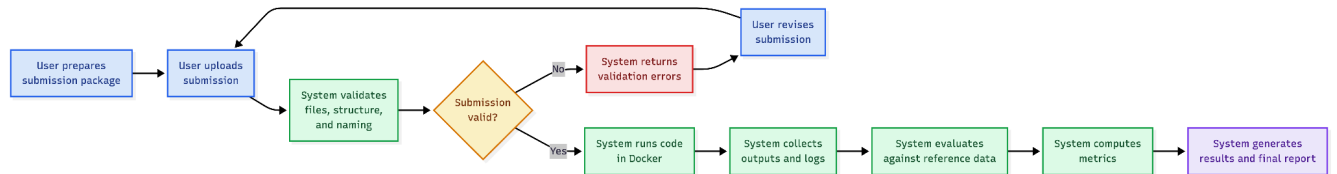
Evaluation Module

The evaluation module will compare submitted outputs against reference datasets. Existing challenge scoring logic will be integrated where possible and extended when needed. The system will compute evaluation metrics such as RMSE, accuracy, repeatability, reproducibility, and related measures.

Reporting Module

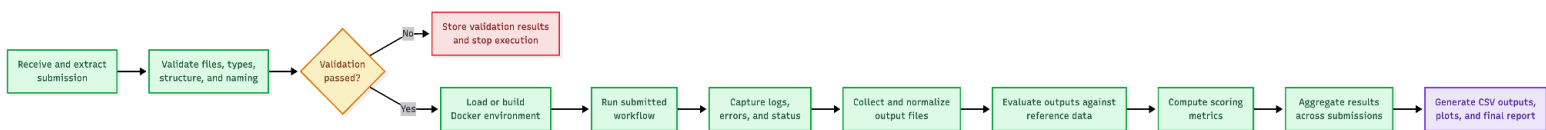
The reporting module will organize results into structured tables, rankings, and visualizations. Outputs will be provided as raw result files and a summary report in HTML or PDF format.

User Flow



The user begins by preparing a submission package that includes parameter maps, metadata, and the associated code. The submission is then uploaded or placed into the required input directory, where the system validates that all necessary files are present and correctly formatted according to the expected structure and naming conventions. If validation fails, the system returns a clear error summary so the user can revise and resubmit their work. If the submission passes validation, the system executes the provided code within an isolated Docker environment to ensure reproducibility. Once execution is complete, the system collects the generated outputs and logs, evaluates the results against reference datasets, and computes relevant performance metrics. Finally, the system produces structured outputs, including rankings, visualizations, and a comprehensive report summarizing the results.

Backend Flow



The backend flow begins when the submission handler receives an input archive or folder and extracts its contents into a working directory for processing. The validation service then checks the submission for required files, correct file types, expected dataset structure, and proper naming conventions. Based on these checks, the system stores the validation results and either stops execution if errors are found or passes the submission to the execution stage. If the submission is valid, the execution service builds or loads the appropriate Docker environment and runs the submitted workflow. During execution, the logging service captures runtime logs, errors, and execution status for monitoring and debugging. After execution, the output parser collects the generated files and converts

them into a normalized format that the evaluation module can process. The evaluation service then compares the outputs against reference data and computes scoring metrics. Finally, the aggregation service stores the results across submissions to generate comparative rankings, and the reporting service produces CSV outputs, plots, and a final HTML or PDF report.

Workflow Summary

The workflow begins when the user submits imaging outputs, metadata, and the associated code. The system then validates the submission to confirm that all required files, formats, and structural requirements are met. If the submission is valid, the submitted workflow is executed in a Docker environment, and the resulting outputs are evaluated against reference data using the defined scoring metrics. Finally, the system generates structured results, including rankings, visualizations, and a final report summarizing performance.

Project Structure

```
project_root/
├── README.md
├── requirements.txt
├── docker/
│   ├── Dockerfile
│   └── docker-compose.yml
├── config/
│   ├── settings.yaml
│   └── validation_rules.yaml
├── data/
│   ├── reference_data/
│   ├── sample_submissions/
│   └── outputs/
├── submissions/
│   ├── incoming/
│   ├── extracted/
│   └── validated/
├── src/
│   └── submission_handler/
│       └── ingest.py
```

```

| | | └─ extract.py
| | └─ validation/
| | | └─ validate_files.py
| | | └─ validate_structure.py
| | | └─ validate_naming.py
| | └─ execution/
| | | └─ run_container.py
| | | └─ capture_logs.py
| | └─ evaluation/
| | | └─ score_submission.py
| | | └─ metrics.py
| | | └─ aggregate_results.py
| | └─ reporting/
| | | └─ generate_tables.py
| | | └─ generate_plots.py
| | | └─ generate_report.py
| | └─ utils/
| | | └─ file_utils.py
| | | └─ logging_utils.py
| | | └─ config_loader.py
└─ tests/
    └─ test_validation.py
    └─ test_execution.py
    └─ test_evaluation.py
    └─ test_reporting.py

```

Deliverables

1. Functional Python evaluation pipeline
2. Automated submission handling and validation system
3. Docker based execution system for submitted workflows
4. Integrated evaluation and scoring module
5. Reporting system with tables, rankings, and visualizations
6. Documentation and example submission workflows

Timeline

The project will be completed over twelve weeks and organized into four main phases: Planning and Setup, Submission and Validation, Execution and Evaluation, and Reporting and Finalization.

Weeks 1 to 2

1. Review existing OSIP challenge workflows and scoring scripts
2. Set up the development environment using Python, Docker, and GitHub
3. Define the input format, expected outputs, and overall pipeline architecture

Weeks 3 to 4

1. Implement submission ingestion for folders and archives
2. Build validation checks for file formats, required files, and naming conventions
3. Add clear validation feedback and error reporting

Weeks 5 to 6

1. Set up the Docker based execution environment
2. Implement workflow execution for submitted code
3. Capture logs, runtime status, and output artifacts

Weeks 7 to 8

1. Integrate challenge scoring logic and reference datasets
2. Compute evaluation metrics such as RMSE, accuracy, repeatability, and reproducibility
3. Validate scoring correctness using sample submissions

Weeks 9 to 10

1. Generate CSV result tables and comparative rankings
2. Create plots and performance visualizations
3. Build the final HTML or PDF reporting system

Weeks 11 to 12

1. Perform end to end testing of the full pipeline
2. Improve performance and handle edge cases
3. Write documentation and example usage guides
4. Prepare the final deliverables.

Gantt Chart: [Link](#)

Risks and Challenges

- Inconsistent or incorrectly formatted submissions may not follow the expected structure, naming conventions, or file formats, leading to validation or execution failures.
- Running user-submitted code introduces security risks, including potentially malicious or unstable scripts.

- Large perfusion imaging datasets can result in increased processing time and higher memory usage.
- Differences in dependencies, environments, and assumptions across submitted workflows can create compatibility issues.

Mitigation Strategies

- Implement strict validation checks for file structure, formats, and naming conventions, with clear error messages to guide users in resolving issues.
- Use Docker to run all submitted code in isolated environments to ensure security, reproducibility, and dependency control.
- Test early and continuously with sample datasets and example submissions to identify issues and ensure reliability.
- Design the system in a modular way so components can be debugged independently and extended without breaking existing functionality.